

PL/SQL Training

SESSION 3: HANDLING EXCEPTIONS & INTERACTING WITH THE ORACLE SERVER

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

SESSION 3

Interacting with the Oracle Server



- Decide which SQL statements can be directly included in a PL/SQL executable block
- Manipulate data with DML statements in PL/SQL
- Make use of the INTO clause to hold the values returned by a SQL statement
- Differentiate between implicit cursors and explicit cursors
- Use SQL cursor attributes

SQL Statements in PL/SQL

- Retrieve a row from the database by using the `SELECT` command.

```
SELECT  select_list
INTO    {variable_name[, variable_name]...
        | record_name}
FROM    table
[WHERE   condition];
```

- The `INTO` clause is required.
 - Queries must return only one row.
- Make changes to rows in the database by using DML commands.
- Control a transaction with the `COMMIT`, `ROLLBACK`, or `SAVEPOINT` command.

Naming Conventions

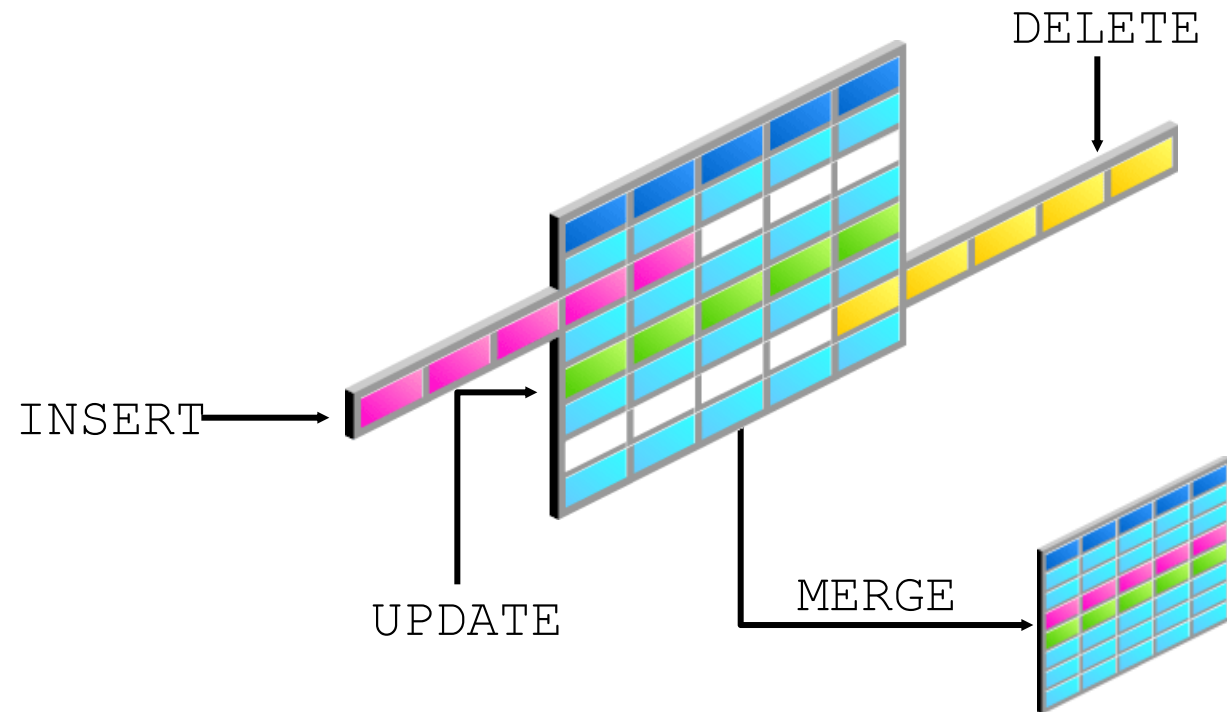


- Use a naming convention to avoid ambiguity in the `WHERE` clause.
- Avoid using database column names as identifiers.
- Syntax errors can arise because PL/SQL checks the database first for a column in the table.
- The names of local variables and formal parameters take precedence over the names of database *tables*.
- The names of database table *columns* take precedence over the names of local variables.

Manipulating Data Using PL/SQL

Make changes to database tables by using DML commands:

- INSERT
- UPDATE
- DELETE
- MERGE



Inserting Data

```
BEGIN

INSERT INTO employees

(employee_id, first_name, last_name, email,hire_date, job_id, salary)

VALUES (employees_seq.NEXTVAL, 'Ruth', 'Cores',

        'RCORES',sysdate, 'AD_ASST', 4000);

COMMIT ;

END;

/
```

Updating Data

```
DECLARE
    sal_increase    employees.salary%TYPE := 800;
BEGIN
    UPDATE          employees
    SET              salary = salary + sal_increase
    WHERE job_id = 'ST_CLERK';
    COMMIT;
END;
/
```

Deleting Data

```
DECLARE
    deptno    employees.department_id%TYPE := 10;
BEGIN
    DELETE FROM    employees
    WHERE    department_id = deptno;
    COMMIT ;
END;
```


A cursor is a pointer to the private memory area allocated by the Oracle server.

There are two types of cursors:

- Implicit cursors: Created and managed internally by the Oracle server to process SQL statements
- Explicit cursors: Explicitly declared by the programmer

SQL Cursor Attributes for Implicit Cursors

Using SQL cursor attributes, you can test the outcome of your SQL statements.

SQL%FOUND	Boolean attribute that evaluates to <code>TRUE</code> if the most recent SQL statement returned at least one row.
SQL%NOTFOUND	Boolean attribute that evaluates to <code>TRUE</code> if the most recent SQL statement did not return even one row.
SQL%ROWCOUNT	An integer value that represents number of rows affected by the most recent SQL statement.

SQL Cursor Attributes for Implicit Cursors

Delete rows that have the specified employee ID from the `employees` table. Print the number of rows deleted.

```
VARIABLE rows_deleted VARCHAR2(30)
DECLARE
    empno employees.employee_id%TYPE := 176;
BEGIN
    DELETE FROM employees
    WHERE employee_id = empno;
    :rows_deleted := (SQL%ROWCOUNT || ' row deleted. ');
END;
/
PRINT rows_deleted
```

SESSION 3

Handling Exceptions

- Define PL/SQL exceptions
- Recognize unhandled exceptions
- List and use different types of PL/SQL exception handlers
- Trap unanticipated errors
- Describe the effect of exception propagation in nested blocks
- Customize PL/SQL exception messages



Type of ERRORS

In Oracle, errors are broadly categorized into several types depending on when and how they occur.

Type	Description
Syntax Errors (Compile-Time Errors)	These occur when the PL/SQL code violates the language's grammar rules — for example, missing keywords, incorrect structure, etc.
Runtime Errors (Exceptions)	These occur during execution when an unexpected condition happens, like dividing by zero, selecting no data, or constraint violations.

Handling Exceptions with PL/SQL

An exception is an error PL/SQL that is raised during program execution.

An exception can be raised:

- Implicitly by the Oracle server
- Explicitly by the program

An exception can be handled:

- By trapping it with a handler
- By propagating it to the calling environment

Exception Types

Implicitly raised

- Predefined Oracle Server
- Non-predefined Oracle Server

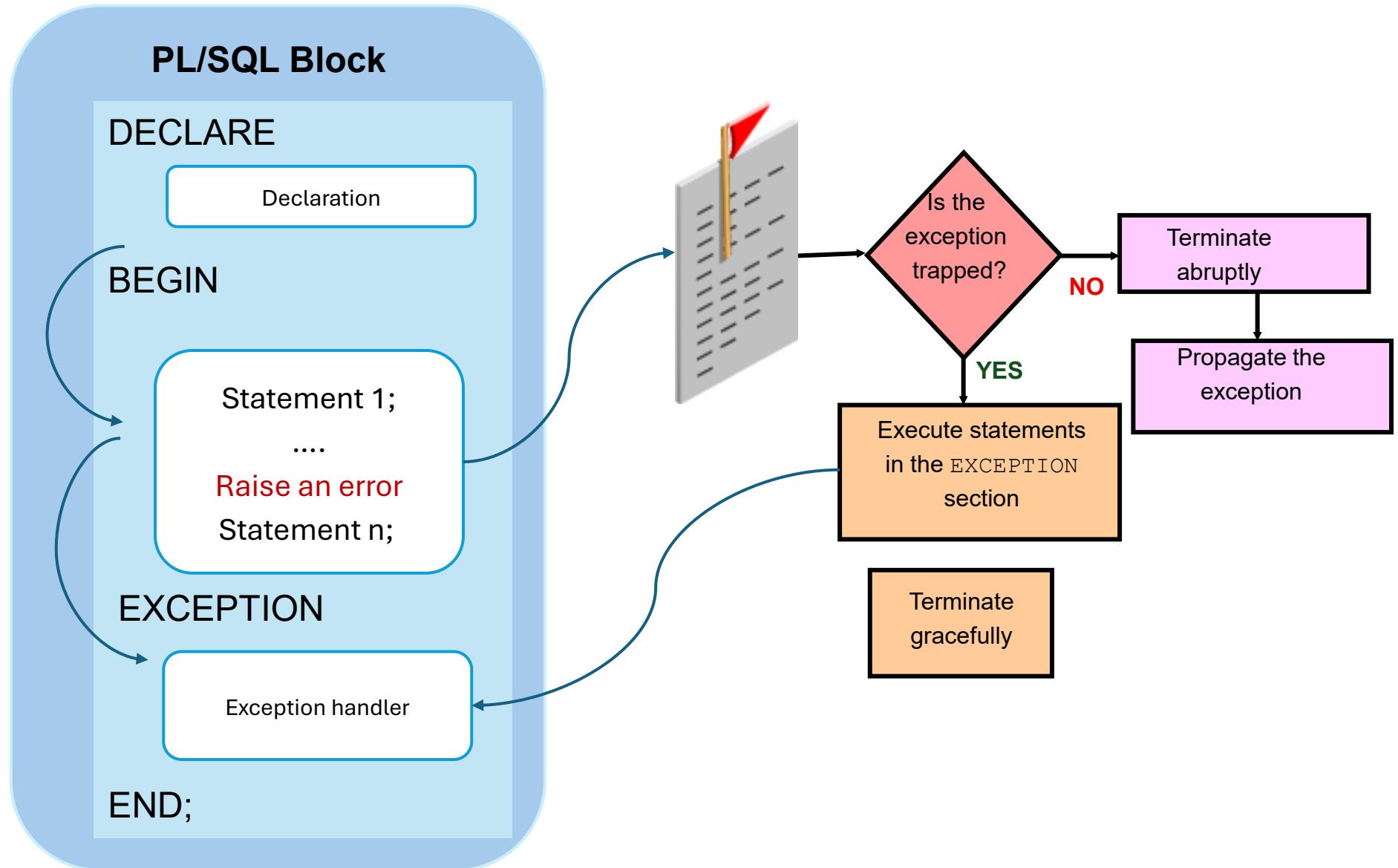
Explicitly raised

- User-defined

Common predefined exceptions

ZERO_DIVIDE	Division by zero
NO_DATA_FOUND	A SELECT INTO query returned no rows
TOO_MANY_ROWS	A SELECT INTO query returned more than one row
VALUE_ERROR	Conversion or assignment error
DUP_VAL_ON_INDEX	Attempt to insert a duplicate value into a unique index

Handling Exceptions with PL/SQL



Trapping Exceptions

- The `EXCEPTION` keyword starts the exception handling section.
- Several exception handlers are allowed.
- Only one handler is processed before leaving the block.
- `WHEN OTHERS` is the last clause.

```
EXCEPTION
  WHEN exception1 [OR exception2 . . .] THEN
    statement1;
    statement2;
    . . .
  [WHEN exception3 [OR exception4 . . .] THEN
    statement1;
    statement2;
    . . .]
  [WHEN OTHERS THEN
    statement1;
    statement2;
    . . .]
```

Functions for Trapping Exceptions

Description	How to get it
The error code. This code is useful when you need to look up generic information about what might cause such a problem.	SQLCODE Note: You cannot call this function inside a SQL statement.
The error message. This text often contains application-specific data such as the name of the constraint or the column associated with the problem.	SQLERRM or DBMS_UTILITY.FORMAT_ERROR_STACK Note: You cannot call SQLERRM inside a SQL statement.
The line on which the error occurred. This capability was added in Oracle Database 10g Release 2 and is enormously helpful in tracking down the cause of errors.	DBMS_UTILITY.FORMAT_ERROR_BACKTRACE
The execution call stack. This answers the question “How did I get here?” and shows you the path through your code to the point at which DBMS_UTILITY.FORMAT_CALL_STACK is called.	DBMS_UTILITY.FORMAT_CALL_STACK

Functions for Trapping Exceptions

SQLCODE:

Returns the numeric value for the error code

SQLERRM:

Returns the message associated with the error number

```
DECLARE
    error_code      NUMBER;
    error_message    VARCHAR2(255);
BEGIN
    ...
EXCEPTION
    ...
    WHEN OTHERS THEN
        ROLLBACK;
        error_code := SQLCODE ;
        error_message := SQLERRM ;

        INSERT INTO errors (e_user, e_date, error_code,
            error_message) VALUES (USER, SYSDATE, error_code,
            error_message);
END;
/
```

Raising exceptions

PL/SQL offers two mechanisms for raising an exception:

- The RAISE statement
 - Syntax: `RAISE VALUE_ERROR;`
- The `RAISE_APPLICATION_ERROR` built-in procedure
 - `RAISE_APPLICATION_ERROR (-ErrorCode , Error message');`

If you need to pass an application-specific message back to your users when an error occurs, you should call the `RAISE_APPLICATION_ERROR` built-in procedure.

This procedure accepts an integer (your error code), whose value must be between -20,999 and -20,000, and a string (your error message).

Summary & Homework



THEORY

[PL/SQL 101 Part 6-7](#)

PRACTICE

[Exceptions](#)

HOMEWORK

[Quiz on Exceptions Raised in Declaration Section](#)

[Quiz on RAISE_APPLICATION_ERROR](#)

TAKEAWAY

[Nine Good-to-Knows for PL/SQL Error Management](#)